Operating Systems Lab - Assignment 7

Consolidated report | xv6 implementation

1. Design decisions and data structures

Q1 - User-level threads. clone(fcn,arg,stack) creates a schedulable thread that shares the caller's address space while retaining a separate process-table entry, kernel stack, and saved trap frame. The user-provided stack is initialized with the argument and a return path to exit(). join() waits for a terminated thread and reclaims its per-thread resources. A shared-address-space reference count prevents one thread from freeing memory still used by its peers; fork() continues to create a private copy. The test starts three workers, each incrementing a shared counter 100,000 times.

Q2 - Semaphores. A fixed table of ten semaphore slots is protected during initialization; each slot stores a count and synchronization lock. sem_wait sleeps while the count is zero and decrements it after waking; sem_signal increments the count and wakes a waiter. The counter test uses a binary semaphore as a mutex. Producer-consumer uses empty=5, full=0, and mutex=1; the producer reserves an empty slot before locking the buffer, and the consumer reserves a full slot before removing an item.

Q3 - Deadlock detection and avoidance. Static Allocation, Request/Need, and Available matrices model processes and resource classes. Detection copies Available into Work and repeatedly finishes a process whose outstanding Request fits Work, then releases that process's Allocation into Work. Any process left unfinished is reported as deadlocked. The Banker safety check derives Need = Max - Allocation and tests whether a complete finish sequence exists before granting a hypothetical request.

Q4 - Priority scheduling. Each process stores a priority from 1 (highest) to 20 (lowest) and a waiting-tick counter. The scheduler chooses the runnable process with the smallest priority number and rotates its scan position to break ties. After 30 waiting ticks, aging improves a process by one priority level, down to priority 1.

Implementation challenges and resolutions

Challenge	Design resolution
Shared address-space lifetime	Track shared ownership and release the page table only when its final thread exits; keep fork copy-on-write behavior out of scope by copying on fork.
Sleeping semaphore waiters	Use a stable sleep channel and a loop around the wait condition so wakeups cannot be mistaken for a granted permit. Keep lock ownership consistent with xv6 sleep/wakeup rules.
Starvation under strict priority	Count runnable wait ticks and age waiting processes every 30 ticks; rotate the scheduler scan to preserve round-robin fairness among equal priorities.
Deadlock test that can hang	Use bounded, reproducible matrix cases; when illustrating a real pipe cycle, isolate it so a watchdog can identify the wait without leaving the shell blocked.

The changes were integrated in the xv6 checkout and the four test programs were added to Makefile under UPROGS.

2. Deadlock tests and race-condition comparison

The matrix tests use two resource types. Values below are representative states supplied to the detection pass. In the cycle case, P0 holds resource A and requests B while P1 holds B and requests A; P2 has no outstanding request and can finish, but cannot release either resource needed by P0 or P1.

Case	Available	Allocation (P0; P1; P2)	Request (P0; P1; P2)	Detection result
Circular wait	[0,0]	[1,0]; [0,1]; [0,0]	[0,1]; [1,0]; [0,0]	P2 finishes; P0 and P1 remain unfinished: deadlock.
Safe state	[1,1]	[1,0]; [0,1]; [0,0]	[0,1]; [1,0]; [0,0]	A finish sequence exists (for example P0, P1, P2): no deadlock.
After unsafe request	[0,0]	[1,0]; [0,1]; [1,1]	[0,1]; [1,0]; [1,1]	Request [1,1] was granted to P2 from Available [1,1]. No process can finish; P0, P1, and P2 remain unfinished.

Banker avoidance check. For a safe state, the algorithm computes Need = Max - Allocation and searches for a sequence in which each remaining Need fits the simulated Work vector. A hypothetical request is tentatively applied only when it is no greater than both the requester's remaining Need and Available. The request is accepted only if the resulting state remains safe; otherwise the tentative allocation is rolled back. The unsafe request example grants P2 a [1,1] request from Available [1,1], leaving [0,0] while P0, P1, and P2 still have outstanding needs. The safety check therefore rejects that request and leaves the prior safe state unchanged.

Real-process illustration. The circular-wait example is: process A retains the write end of Pipe 1 while waiting to read Pipe 2; process B retains the write end of Pipe 2 while waiting to read Pipe 1. Each wait depends on the other process producing data. This forms the same A-to-B-to-A wait cycle represented by the matrix test. A bounded supervisor/watchdog is used so the demonstration can be terminated safely.

Q1 versus Q2: counter results

Run	Critical-section protection	Expected final counter	Interpretation
Q1 threadtest	None	Usually less than 300,000; varies by scheduling	The increment is a read-modify-write operation. Interleaving can overwrite another thread's update, so lost increments are expected.
Q2 synctest	sem_wait / increment / sem_signal	300,000	The semaphore serializes the update; all three workers contribute 100,000 increments and the final count is exact.

The Q1 number is nondeterministic and may vary between runs; 300,000 is the target absent lost updates, not a guaranteed unsynchronized result. Q2 should consistently reach 300,000 when every worker completes and the semaphore is shared correctly.

Producer-consumer checks use the invariant 0 <= full <= 5 and empty + full = 5. Each produced item is paired with one consumed item; the empty/full semaphores prevent overflow and underflow, while the mutex protects the buffer indices and contents.

3. Priority-scheduling results and verification

The priority test forks four CPU-bound children at priorities 1, 5, 10, and 20. Under priority scheduling, the smaller numeric priority should receive CPU time sooner and tend to finish earlier. Aging gives a continuously runnable low-priority process a better priority after each 30-tick wait interval, so it eventually competes with higher-priority work. Completion order can vary with timer granularity, workload, and emulator speed; the table expresses the expected qualitative trend rather than universal tick measurements.

Initial priority	Expected relative completion time	Expected aging / starvation behavior
1	Shortest	Selected ahead of lower-priority runnable processes; remains at best priority.
5	Short	May wait behind priority 1; tie-breaking remains round-robin.
10	Intermediate	Aging improves priority after 30 runnable wait ticks.
20	Longest initially	Wait ticks trigger repeated promotion toward 1; eventual completion demonstrates starvation prevention.

For a measured completion-time graph/table, record each child's elapsed timer ticks at exit along with its initial and final priority. Exact elapsed-tick values depend on the xv6 run and are intentionally not replaced with fabricated measurements here. The completion-order output and printed aging transitions show whether the observed run follows the expected trend.

Verification summary

Part	Evidence to record from the xv6 run	Pass condition
Q1 threads	clone/join logs and final counter	Three workers complete and the shared counter is visible; unsynchronized result may be below 300,000.
Q2 semaphores	Counter value and produced/consumed item log	Protected counter is 300,000; buffer indices stay in range and each item is consumed once.
Q3 detection	Available, Allocation, Request/Need, finish sequence	Cycle case identifies P0/P1; safe state finishes all; unsafe post-request state leaves processes unfinished.
Q4 scheduler	Completion order, elapsed ticks, aging trace	Completion generally tracks priority and priority-20 worker advances through aging and completes.

Summary

The implementation combines shared-memory threads, blocking synchronization, matrix-based deadlock analysis, and priority scheduling with aging. The tests are designed to make the key properties observable: shared state across threads, exact protected increments, correct safe/unsafe classification, bounded producer-consumer behavior, and eventual progress for low-priority work. Runtime values should be interpreted with the xv6 console output from the submitted build.